

手摸手实现基础组件模块

组件地址

```
<dependency>
  <groupId>org.opengoofy.index12306</groupId>
  <artifactId>index12306-base-spring-boot-starter</artifactId>
  <version>${project.version}</version>
</dependency>
```

`index12306-base-spring-boot-starter` 在 12306 的整体规划中，属于顶层组件或基础组件。

那什么样的组件能被称为顶层？可以从几个方面去说明这个问题：

- 兼容性：能够被所有组件或者所有客户端项目所依赖，不会存在兼容性的代码组件。
- 功能性：独立且公用的功能可被用户无感知依赖。比如当 fastjson 出现安全漏洞，可通过升级顶层组件来解决所有依赖包问题。

组件概览

1. 定义全局配置常量和过滤器执行顺序。
2. 封装 Spring 应用上下文 `ApplicationContextHolder`。
3. 封装 FastJSON 安全模式，客户端可通过配置 `fastjson.safer-mode=true` 开启安全模式。
4. 封装应用初始化事件，通过 Spring `ApplicationReadyEvent` 进行发布，并保证仅执行一次，`ApplicationInitializingEvent`。
5. 单例对象容器，可避免重复创建对象，方便全局访问。

组件功能

全局变量&组件执行顺序

我们先来看下，全局变量&组件执行顺序功能在代码中的体现都是什么？

1) 全局用户常量。

```
package org.opengoofy.index12306.framework.starter.bases.constant;

/**
 * 用户常量
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
public final class UserConstant {

    /**
     * 用户 ID Key
     */
    public static final String USER_ID_KEY = "userId";

    /**
     * 用户名 Key
     */
    public static final String USER_NAME_KEY = "username";

    /**
     * 用户真实名称 Key
     */
    public static final String REAL_NAME_KEY = "realName";
}
```

思考下，为什么要定义这个全局变量定义常量类？\ 如果没有定义在顶级组件库中，那么这些常量都是定义在各个组件库中的。比如用户的常量定义在用户的组件库中。

那又为什么把这个抽象提炼出来？

一旦涉及到抽象提炼的工作，就能说明，这个常量肯定不是只在用户组件库中使用。因为在网关中，将用户 Token 进行解析，并放到 HTTP Header 中，最终放到用户请求上下文，也需要用到这些用户常量。

```
@Override
public GatewayFilter apply(Config config) {
    return (exchange, chain) -> {
        ServerHttpRequest request = exchange.getRequest();
```

```

String requestPath = request.getPath().toString();
if (isPathInBlackPreList(requestPath, config.getBlackPathPre())) {
    String token = request.getHeaders().getFirst("Authorization");
    UserInfoDTO userInfo = JWTUtil.parseJwtToken(token);
    if (!validateToken(userInfo)) {
        ServerHttpResponse response = exchange.getResponse();
        response.setStatusCode(HttpStatus.UNAUTHORIZED);
        return response.setComplete();
    }

    ServerHttpRequest.Builder builder =
exchange.getRequest().mutate().headers(httpHeaders -> {
    // 通过 HTTP Token 获取用户 ID 放入 HTTP Header 中
    httpHeaders.set(UserConstant.USER_ID_KEY, userInfo.getUserId());
    // 同上
    httpHeaders.set(UserConstant.USER_NAME_KEY, userInfo.getUsername());
    try {
        // 同上
        httpHeaders.set(UserConstant.REAL_NAME_KEY,
URLLEncoder.encode(userInfo.getRealName(), "UTF-8"));
    } catch (UnsupportedEncodingException e) {
        throw new RuntimeException(e);
    }
});
return chain.filter(exchange.mutate().request(builder.build()).build());
}
return chain.filter(exchange);
};
}

```

类似于这种多处使用的变量，分别定义肯定是不优雅的，那我们只能找个地方将这个定义，并被用户和网关同时使用。基础组件库因为要被每个组件库依赖，所以是最好的选择。

2) 全局过滤器顺序执行常量类

```

package org.opengoofy.index12306.framework.starter.bases.constant;

/**
 * 全局过滤器顺序执行常量类
 *

```

```
* @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
*/
public final class FilterOrderConstant {

    /**
     * 用户信息传递过滤器执行顺序排序
     */
    public static final int USER_TRANSMIT_FILTER_ORDER = 100;
}
```

再思考下，过滤器、拦截器或 AOP 组件的执行顺序又为什么定义在基础组件库？

提个问题：假设你的应用依赖了日志打印和幂等组件，方法执行时，需要打印日志和避免请求幂等。流程伪代码如下：

```
@Log
@Idempotent
private boolean validateToken(UserInfoDTO userInfo) {
    return userInfo != null;
}
```

正常来说，要先打印日志，再触发幂等行为。但是，如果说日志和幂等的组件库开发者不是同一人，幂等在日志请求之前。如果不触发幂等还好，一旦触发幂等行为抛出异常，那么日志组件将不再执行。这对于生产日志以及排查问题来说非常不友好。

所以说，每个基础组件的执行顺序需要在全局定义，每个组件开发者要有全局思维，定义类似于这种过滤器或拦截器又或者 AOP 时，需要从组件功能再结合全局组件功能考虑到执行顺序问题。

正因为如此，将这些组件的执行顺序单独放到基础组件库中，一个是基础组件库被全部组件库依赖，另一个就是定义组件时，也能看到其他组件库的功能以及执行顺序。避免出现信息孤岛问题。

总得来说，有以下这些优势：

- 解耦和可复用性：通过将组件库的执行顺序抽象到一个基础的组件库中，可以将各个组件库之间的依赖和执行顺序解耦。这样，每个组件库可以独立开发、测试和维护，提高了代码的可复用性和可维护性。

- 灵活性：由于执行顺序被抽象到基础组件库中，可以根据具体的业务需求对组件库进行配置，灵活地调整组件的执行顺序。这种灵活性使得开发人员可以根据具体的场景和需求来定义过滤器的执行顺序，提供更好的定制化和灵活性。
- 执行顺序的统一管理：将执行顺序抽象到一个基础组件库中，可以实现对所有组件库中过滤器执行顺序的统一管理。开发人员只需关注各个组件库的业务逻辑实现，而不必关心过滤器的具体执行顺序。这样可以简化开发流程，提高开发效率。
- 扩展性：基础组件库可以提供一些通用的功能和特性，例如全局变量的管理、过滤器的注册和执行等。这样可以为其他组件库提供扩展性，使其能够更好地适应不同的业务场景和需求。

Spring 容器上下文

很多时候，我们也需要在非 Spring Bean 中使用到 Spring Bean。

如果说在 Spring Bean 中获取 Spring Bean，只需要通过构造器或者 @Autowired 或 @Resource 就可以了。但是非 Spring Bean 却不行。

举个简单的例子，大家定义线程池任务实现类时，可能需要获取到 Spring Bean。

基于以上诉求，我们依赖 Spring 提供的 ApplicationContextAware 接口，来将 Spring IOC 容器的对象放到一个自定义容器中，并持有 Spring IOC 容器。

这样就可以通过自定义容器访问 Spring IOC 容器获取 Spring Bean。

```
package org.opengoofy.index12306.framework.starter.bases;

import org.springframework.beans.BeansException;
import org.springframework.context.ApplicationContext;
import org.springframework.context.ApplicationContextAware;

import java.lang.annotation.Annotation;
import java.util.Map;

/**
 * Application context holder.
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
```

```

public class ApplicationContextHolder implements ApplicationContextAware {

    private static ApplicationContext CONTEXT;

    @Override
    public void setApplicationContext(ApplicationContext applicationContext) throws
BeansException {
        ApplicationContextHolder.CONTEXT = applicationContext;
    }

    /**
     * Get ioc container bean by type.
     *
     * @param clazz
     * @param <T>
     * @return
     */
    public static <T> T getBean(Class<T> clazz) {
        return CONTEXT.getBean(clazz);
    }

    /**
     * Get ioc container bean by name and type.
     *
     * @param name
     * @param clazz
     * @param <T>
     * @return
     */
    public static <T> T getBean(String name, Class<T> clazz) {
        return CONTEXT.getBean(name, clazz);
    }

    /**
     * Get a set of ioc container beans by type.
     *
     * @param clazz
     * @param <T>
     * @return
     */
    public static <T> Map<String, T> getBeansOfType(Class<T> clazz) {

```

```

        return CONTEXT.getBeansOfType(clazz);
    }

    /**
     * Find whether the bean has annotations.
     *
     * @param beanName
     * @param annotationType
     * @param <A>
     * @return
     */
    public static <A extends Annotation> A findAnnotationOnBean(String beanName, Class<A>
annotationType) {
        return CONTEXT.findAnnotationOnBean(beanName, annotationType);
    }

    /**
     * Get ApplicationContext.
     *
     * @return
     */
    public static ApplicationContext getInstance() {
        return CONTEXT;
    }
}

```

FastJSON 安全模式

Fastjson 的 "autoType" 特性是指在反序列化过程中，允许将 JSON 字符串自动转换为指定的 Java 类型。它提供了一种方便的方式，使得开发人员可以直接将 JSON 数据转换为相应的 Java 对象，而无需手动指定目标类。

然而，这个特性也存在一定的安全风险。攻击者可以构造恶意的 JSON 数据，其中包含对不受信任的类的引用。当 "autoType" 特性被启用时，Fastjson 会尝试根据 JSON 字符串中的类信息实例化相应的对象，从而可能导致潜在的安全问题，例如远程代码执行攻击。

为了说明这个问题，考虑以下示例：

```
String jsonString = "{\"@type\":\"com.example.EvilClass\",\"data\":\"Malicious Data\"}";
```

```
Object obj = JSON.parseObject(jsonString);
```

在上述示例中，JSON 字符串中的 "@type" 属性指定了一个名为 "com.example.EvilClass" 的类。如果 Fastjson 的 "autoType" 特性被启用，它将尝试将该 JSON 字符串转换为 "com.example.EvilClass" 类的实例，而不考虑该类是否可信或预期。

为了防止这种安全问题，Fastjson 默认情况下禁用了 "autoType" 特性，以提供一定的安全性。但是，在某些特定场景下，开发人员可能会手动启用该特性，例如在处理受信任的 JSON 数据时需要转换为多态对象。

我们 FastJSON 安全模式，会关闭该 autoType 特性。

```
package org.opengoofy.index12306.framework.starter.bases.safa;

import org.springframework.beans.factory.InitializingBean;

/**
 * FastJson安全模式，开启后关闭类型隐式传递
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
public class FastJsonSafeMode implements InitializingBean {

    @Override
    public void afterPropertiesSet() throws Exception {
        System.setProperty("fastjson2.parser.safeMode", "true");
    }
}
```

如果配置文件中配置了 `framework.fastjson.safa-mode=true` 那么则开启安全模式，反之无任何变化。

```
package org.opengoofy.index12306.framework.starter.bases.config;

import org.opengoofy.index12306.framework.starter.bases.ApplicationContextHolder;
import
org.opengoofy.index12306.framework.starter.bases.init.ApplicationContentPostProcessor;
import org.opengoofy.index12306.framework.starter.bases.safa.FastJsonSafeMode;
import org.springframework.boot.autoconfigure.condition.ConditionalOnMissingBean;
```



```
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.Bean;

/**
 * 应用基础自动装配
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
public class ApplicationBaseAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean
    @ConditionalOnProperty(value = "framework.fastjson.safa-mode", havingValue = "true")
    public FastJsonSafeMode congoFastJsonSafeMode() {
        return new FastJsonSafeMode();
    }
}
```

安全初始化事件

我们在实际应用开发中，会依赖很多应用初始化时执行任务，比如说 `InitializingBean`、`CommandLineRunner` 等。

除此之外，还有一些比如依赖 Spring 初始化的一些事件。

1. `ContextRefreshedEvent`：当应用程序上下文（`ApplicationContext`）初始化或刷新完成后触发。这通常发生在应用程序启动过程中，并且表示应用程序已准备好接收请求和执行业务逻辑。可以使用该事件来执行一些初始化操作，例如加载缓存数据、启动后台任务等。
2. `ContextStartedEvent`：当应用程序上下文启动时触发。这个事件在调用 `ConfigurableApplicationContext` 的 `start()` 方法后被发布。可以使用该事件来执行启动应用程序所需的特定逻辑，例如启动定时任务、启动消息监听器等。
3. `ContextStoppedEvent`：当应用程序上下文停止时触发。这个事件在调用 `ConfigurableApplicationContext` 的 `stop()` 方法后被发布。可以使用该事件来执行停止应用程序所需的清理逻辑，例如关闭连接、释放资源等。
4. `ContextClosedEvent`：当应用程序上下文关闭时触发。这个事件在调用 `ConfigurableApplicationContext` 的 `close()` 方法后被发布。可以使用该事件来执行一些最终的清理操作，例如释放数据库连接、销毁单例对象等。

5. `ServletRequestHandledEvent`: 当 Spring MVC 处理完一个 HTTP 请求时触发。该事件提供了有关请求处理的详细信息, 包括请求的处理时间、处理器、处理器适配器等。可以使用该事件来进行请求处理的监控、日志记录或统计信息收集等操作。
6. `ApplicationStartedEvent` (Spring Boot): 当 Spring Boot 应用程序启动时触发。这个事件在 `SpringApplication.run()` 方法完成后被发布。可以使用该事件来执行特定于应用程序的初始化逻辑。
7. `ApplicationReadyEvent` (Spring Boot): 当 Spring Boot 应用程序准备就绪时触发。这个事件表示应用程序已经启动完毕, 并且已经可以提供服务。可以使用该事件来执行应用程序的后续初始化操作, 例如加载数据、发送通知等。

有些场景是依赖 Spring 容器初始化完成后调用的, `ContextRefreshedEvent` 这个时间就比较合适。但是大家发现没有, 它除了初始化调用, 容器刷新也会调用。

为了避免容器刷新造成二次调用初始化逻辑, 我们对一些比较常用的事件简单封装了一层逻辑。

1) 首先定义初始化事件对象类。

```
package org.opengoofy.index12306.framework.starter.bases.init;

import org.springframework.context.ApplicationEvent;

/**
 * 应用初始化事件
 *
 * <p> 规约事件, 通过此事件可以查看业务系统所有初始化行为
 *
 * @公众号: 马丁玩编程, 回复: 加群, 添加马哥微信 (备注: 12306) 获取项目资料
 */
public class ApplicationInitializingEvent extends ApplicationEvent {

    /**
     * Create a new {@code ApplicationEvent}.
     *
     * @param source the object on which the event initially occurred or with
     *               which the event is associated (never {@code null})
     */
    public ApplicationInitializingEvent(Object source) {
        super(source);
    }
}
```

```
}  
}
```

2) 其次，定义应用初始化后置处理器，防止 Spring 事件被多次执行。

整体逻辑也比较简单，通过锁来保证同一时间只有一个事件进行初始化。初始化后设置一个标识，下次如果再触发，就不再执行，类似于幂等处理的逻辑。

```
package org.opengoofy.index12306.framework.starter.bases.init;  
  
import lombok.RequiredArgsConstructor;  
import org.springframework.boot.context.event.ApplicationReadyEvent;  
import org.springframework.context.ApplicationContext;  
import org.springframework.context.ApplicationListener;  
  
/**  
 * 应用初始化后置处理器，防止Spring事件被多次执行  
 *  
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料  
 */  
@RequiredArgsConstructor  
public class ApplicationContentPostProcessor implements  
    ApplicationListener<ApplicationReadyEvent> {  
  
    private final ApplicationContext applicationContext;  
  
    /**  
     * 执行标识，确保Spring事件 {@link ApplicationReadyEvent} 有且执行一次  
     */  
    private boolean executeOnlyOnce = true;  
  
    @Override  
    public void onApplicationEvent(ApplicationReadyEvent event) {  
        synchronized (ApplicationContentPostProcessor.class) {  
            if (executeOnlyOnce) {  
                applicationContext.publishEvent(new ApplicationInitializingEvent(this));  
                executeOnlyOnce = false;  
            }  
        }  
    }  
}
```

```
}  
}
```

单例对象容器

提供一种单例的访问模式，单例的优点如下：

1. 全局访问：单例对象可以在应用程序的任何地方被访问，而不需要传递对象的引用。这样可以方便地共享对象的状态和功能，简化了对象之间的通信和协作。
2. 节省资源：由于只有一个对象实例存在，可以减少重复创建对象的开销。在需要频繁创建和销毁对象的情况下，单例对象可以显著节省系统资源，提高性能。

应用场景也比较多，比如说，发送邮件时需要加载邮件的模板文件，重复加载就比较浪费性能。完全可以加载一次后，将该对象放到单例对象容器中缓存，下次使用时直接获取就好。

```
package org.opengoofy.index12306.framework.starter.bases;  
  
import lombok.AccessLevel;  
import lombok.NoArgsConstructor;  
  
import java.util.concurrent.ConcurrentHashMap;  
import java.util.function.Supplier;  
  
/**  
 * 单例对象容器  
 *  
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料  
 */  
@NoArgsConstructor(access = AccessLevel.PRIVATE)  
public final class Singleton {  
  
    private static final ConcurrentHashMap<String, Object> SINGLE_OBJECT_POOL = new  
        ConcurrentHashMap();  
  
    /**  
     * 根据 key 获取单例对象  
     */  
    public static <T> T get(String key) {  
        Object result = SINGLE_OBJECT_POOL.get(key);  
        return result == null ? null : (T) result;  
    }  
}
```

```

}

/**
 * 根据 key 获取单例对象
 *
 * <p> 为空时，通过 supplier 构建单例对象并放入容器
 */
public static <T> T get(String key, Supplier<T> supplier) {
    Object result = SINGLE_OBJECT_POOL.get(key);
    if (result == null && (result = supplier.get()) != null) {
        SINGLE_OBJECT_POOL.put(key, result);
    }
    return result != null ? (T) result : null;
}

/**
 * 对象放入容器
 */
public static void put(Object value) {
    put(value.getClass().getName(), value);
}

/**
 * 对象放入容器
 */
public static void put(String key, Object value) {
    SINGLE_OBJECT_POOL.put(key, value);
}
}

```

更新: 2023-07-17 12:24:22

原文: <<https://www.yuque.com/magestack/12306/gmk40bad5dickgul>>